

Apache Airflow

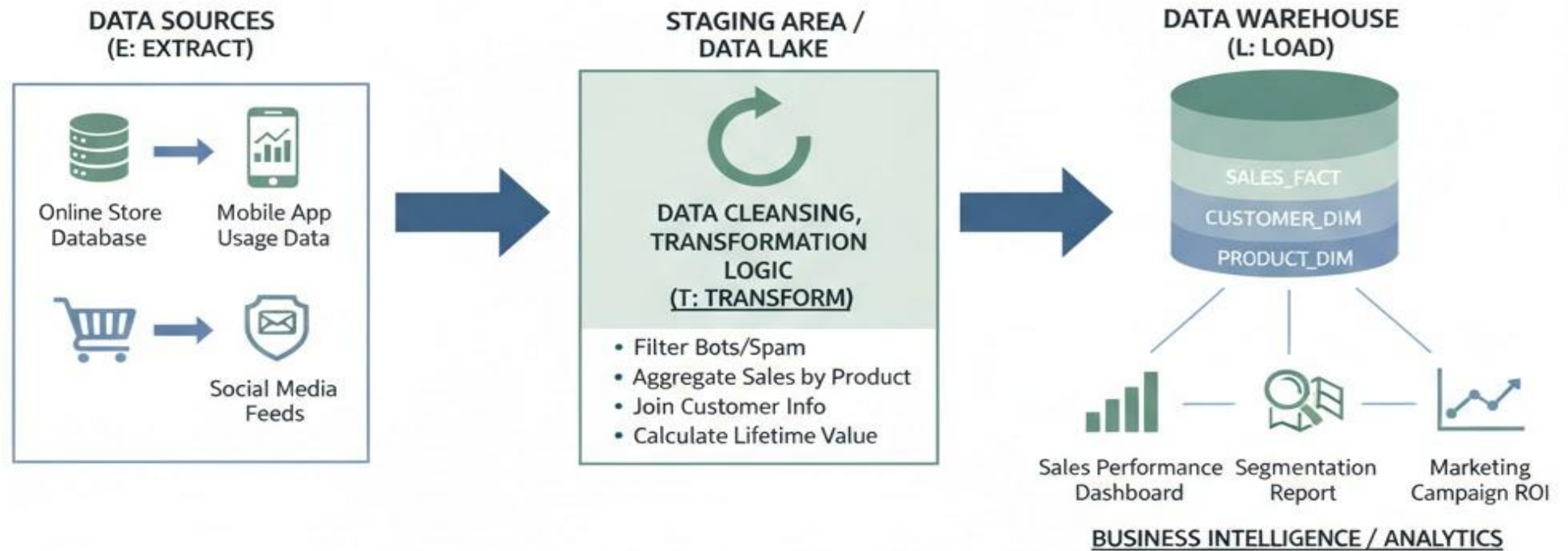
Author, Schedule and Monitor workflows

Airflow

- Apache Airflow is an open-source platform used for programmatically author, schedule and monitor workflows.
- Airflow is commonly used for data pipelines where tasks like Data **Extraction, Transformations and Loading (ETL)** are orchestrated across multiple systems.
- Pipeline example is extracting orders and payments from database (remote/local)(**Extract**) remove cancelled orders (**Transform**) and store cleaned data to database/cloud(**Load**).

Airflow

ETL PIPELINE FOR E-COMMERCE ANALYTICS



Airflow

Apache Airflow can be used to schedule:

- ETL pipelines that extract data from multiple sources and run Spark jobs or any other data transformations
- Training machine learning models after deployment
- Report generation
- Backups and similar DevOps operations

Key Components of Airflow

Airflow has many key components:

- a) DAG (Directed Acyclic Graphs)
- b) Task
- c) Operators
- d) Schedulers
- e) Executors
- f) WebServer
- g) Database

1. DAG

- Also called as **The Entire Workflow / Pipeline**
- A DAG is a blueprint of your workflow—it defines **what tasks run, in what order, and how often.**

```
from airflow import DAG
from datetime import datetime
```

```
dag = DAG(
    dag_id="simple_dag",
    start_date=datetime(2024, 1, 1),
    schedule="@daily"           # It runs daily.
)
```

2. Task

- Task is a single step in the Workflow.
- Each task performs **one action**.

```
from airflow.operators.python import PythonOperator
def greet():
    print("Hello from Airflow Task!") # Callable Function
# Task 1
task = PythonOperator(
    task_id="greet_task",
    python_callable=greet,
    dag=dag,
)
```

3. Operator

1/2

- What Type of Task It Is?
- An **operator** defines *how* the task runs (what action it performs).
- Each operator has a different job.

Operator	What It Does
PythonOperator	Run Python code
BashOperator	Run shell commands
EmailOperator	Send emails
S3ToGCSOperator	Move data
DockerOperator	Run tasks in Docker
KubernetesPodOperator	Run tasks in K8s pod

3. Operator

2/2

```
from airflow.operators.bash import BashOperator
```

```
bash_task = BashOperator(  
    task_id="show_date",  
    bash_command="date",  
    dag=dag  
)
```

4. Scheduler

- The **scheduler** monitors time and sends tasks for execution when it's time.
- Airflow scheduler tells tasks: "It's 12 AM — run your DAG now!"

```
dag = DAG(  
    dag_id="ml_training_daily",  
    schedule="@daily" )
```

i.e @daily, @weekly, @monthly, @hourly, @yearly

schedule="30 8 * * 1-5" will run every weekday (Mon-Fri) at 8:30 AM.

5. Executer

- How Tasks Are Executed?
- The **executor** decides *HOW* tasks run (locally, on multiple workers, in Kubernetes, etc.).

Executor	Where It Runs	When to Use
SequentialExecutor	1 process	Beginner / local
LocalExecutor	Multiple parallel processes	Small teams
CeleryExecutor	Distributed workers	Enterprise workflows
KubernetesExecutor	Kubernetes Pods	MLOps / heavy workloads

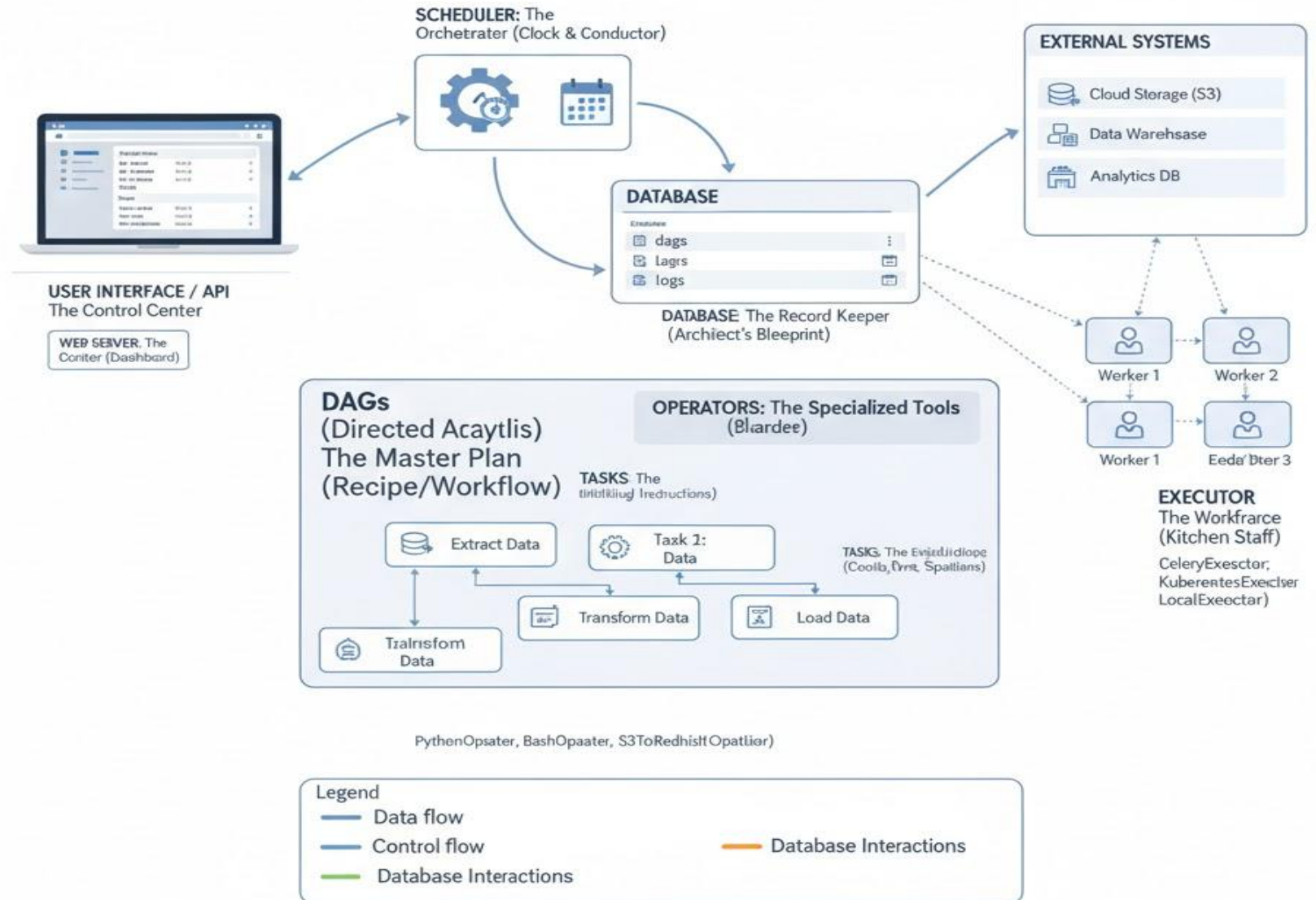
6. Webserver

- The Airflow UI
- The **webserver** runs the UI where you can:
 - see DAGs
 - trigger runs
 - check logs
 - view graphs
 - monitor pipeline health

Airflow

APACHE AIRFLOW KEY COMPONENTS

Real-World E-commerce Data Pipeline Analogy



Airflow Setup

Two options

1. Standalone
2. Astronomer Airflow

Standalone Airflow

```
pip install --upgrade pip setuptools wheel
```

```
pip install apache-airflow
```

```
# Windows PowerShell:
```

```
# $env:AIRFLOW_HOME = "$PWD\airflow_home"
```

Astronomer with Airflow

Install Astro

1. `wsl --update`
2. `wsl --install --no-distribution`
3. `winget install -e --id Astronomer.Astro`

Astronomer with Airflow

Important Commands:

- a) `astro dev init` (Create Project)
- b) `astro dev start`
- c) `astro dev stop`